

# CMPE343 Project Report

Group: c-sharp

Student 002 (student\_002)

Student 003 (student\_003)

Student 001 (student\_001)

<https://github.com/bouncmpe343/project-c>

December 2025

## Contents

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                             | <b>2</b>  |
| <b>2</b> | <b>Part 1: Conditional Probability Modeling</b> | <b>2</b>  |
| 2.1      | Problem Description . . . . .                   | 2         |
| 2.2      | Methodology & Implementation . . . . .          | 2         |
| 2.3      | Results . . . . .                               | 4         |
| 2.4      | Discussion . . . . .                            | 9         |
| <b>3</b> | <b>Part 2: User Variability Modeling</b>        | <b>10</b> |
| 3.1      | Problem Description . . . . .                   | 10        |
| 3.2      | Methodology & Implementation . . . . .          | 11        |
| 3.3      | Results . . . . .                               | 13        |
| 3.4      | Discussion . . . . .                            | 15        |
| <b>4</b> | <b>Part 3: Recommender Design</b>               | <b>16</b> |
| 4.1      | Problem Description . . . . .                   | 16        |
| 4.2      | Model Definitions (Mathematical) . . . . .      | 16        |
| 4.3      | Probabilistic Reasoning (Parts 1 & 2) . . . . . | 17        |
| 4.4      | Methodology & Implementation . . . . .          | 17        |
| 4.5      | Results . . . . .                               | 17        |
| 4.6      | Discussion . . . . .                            | 18        |
| <b>5</b> | <b>Part 4: Monte Carlo Evaluation</b>           | <b>18</b> |
| 5.1      | Problem Description . . . . .                   | 18        |
| 5.2      | Data and Enlargement . . . . .                  | 18        |
| 5.3      | Methodology & Implementation . . . . .          | 18        |
| 5.4      | Results . . . . .                               | 19        |
| 5.5      | Interpretation . . . . .                        | 20        |
| 5.6      | Why Top-1 and Hit@3 . . . . .                   | 20        |
| 5.7      | Discussion . . . . .                            | 20        |
| <b>6</b> | <b>Conclusion</b>                               | <b>20</b> |
| 6.1      | Key Findings . . . . .                          | 20        |

# 1 Introduction

This project studies how probabilistic modeling improves music recommendation. Using real user ratings and track metadata, we build models to estimate the probability of high ratings, analyze user variability, and design recommenders that move beyond a one-size-fits-all baseline. Our objectives are to (1) quantify feature-driven preferences, (2) capture differences in user pickiness, (3) implement a popularity baseline and a personalized recommender grounded in these insights, and (4) evaluate both models with a Monte Carlo replay to measure hit rate, rating quality, and time-to-favorite. The motivation is practical: better personalization should surface relevant tracks earlier while remaining robust for sparse histories.

## 2 Part 1: Conditional Probability Modeling

### 2.1 Problem Description

In this part, we aim to estimate the probability of a song receiving a 5-star rating based on various features using both global user data and personal session data. The specific tasks are as follows:

1. **Global Conditional Probabilities:** Estimate  $P(5\star|\text{Feature})$  for features such as Artist, Year, and Explicit content, incorporating Laplace smoothing.
2. **Feature Interactions:** Analyze combinations of features (e.g., Artist + Genre) to see if they predict preferences better than single features.
3. **Bayesian Interpretation:** Use Bayes' rule to invert probabilities and calculate  $P(\text{Feature}|5\star)$  to identify characteristics that dominate high-rated songs.
4. **Personal and Group Analysis:** Compute personal probabilities  $P_m(5\star|F)$  and aggregate them to form a group-level model  $P_{\text{group}}(5\star|F)$ .

### 2.2 Methodology & Implementation

We implemented the probability modeling in `src/part1.py`. The core logic relies on the following functions:

- `calculate_conditional_prob`: Calculates  $P(\text{Target}|\text{Feature})$  using Laplace smoothing. It returns counts, hits, and probabilities.

```

1  def calculate_conditional_prob(df: pd.DataFrame, feature_col: str,
2      target_col: str = 'rating', target_val: int = 5, alpha: int = 1, k: int
3      = 2) -> pd.DataFrame:
4      """
5          Calculates P(Target=target_val | Feature) with Laplace smoothing.
6          Returns a DataFrame containing counts, priors, and conditional
7          probs.
8          """
9      # 1. Group by Feature and Calculate Counts (Denominator: Count(F))
10     feature_counts = df[feature_col].value_counts()
11
12     # 2. Count occurrences where Target is met (Numerator: Count(Target
    and Feature))
13     target_hits = df[df[target_col] == target_val][feature_col].
    value_counts()
14
15     # Reindex to ensure indices match, filling missing intersections
    with 0

```

```

13     target_hits = target_hits.reindex(feature_counts.index, fill_value
14     =0)
15
16     # 3. Apply Laplace Smoothing
17     probs = laplace_smooth(target_hits, feature_counts, alpha, k)
18
19     # 4. Calculate Prior P(Feature) for Bayes Rule
20     p_feature = feature_counts / len(df)
21
22     # Create result DataFrame
23     results = pd.DataFrame({
24         'feature_value': feature_counts.index,
25         'count': feature_counts.values,
26         'hits': target_hits.values,
27         f'P({target_val}*|Feature)': probs.values,
28         'P(Feature)': p_feature.values
29     })
30
31     return results

```

Listing 1: Calculate Conditional Probability

- `laplace_smooth`: A helper function satisfying the smoothing formula:  $\frac{\text{hits} + \alpha}{\text{total} + \alpha \cdot k}$ .
- `analyze_feature_prob`: A wrapper to calculate and display top probabilities for a single feature.
- `analyze_interaction`: Creates a combined feature column (e.g., "Happy + Bright") and runs probability analysis on it.
- `calculate_bayes_inverse`: Applies Bayes' theorem to find  $P(\text{Feature}|\text{Target})$  using the global prior  $P(\text{Target})$ .

```

1     def calculate_bayes_inverse(prob_df: pd.DataFrame, df: pd.DataFrame,
2     target_rating: int = 5) -> pd.DataFrame:
3         """
4         Applies Bayes' Theorem to invert probabilities.
5         Calculates P(Feature | Target) = P(Target | Feature) * P(Feature) /
6         P(Target)
7         """
8         if prob_df is None: return None
9
10        # Calculate Global P(Target)
11        total_target_hits = len(df[df['rating'] == target_rating])
12        p_target_global = total_target_hits / len(df)
13
14        if p_target_global == 0:
15            print("Global probability of target is 0, cannot invert.")
16            return prob_df
17
18        # Copy to avoid SettingWithCopy warnings
19        bayes_df = prob_df.copy()
20
21        # P(F | T) = (P(T | F) * P(F)) / P(T)
22        cond_col = f'P({target_rating}*|Feature)'
23        bayes_col = f'P(Feature|{target_rating}*)'
24
25        bayes_df[bayes_col] = (bayes_df[cond_col] * bayes_df['P(Feature)'])
26        / p_target_global
27
28        print(f"\n--- Bayesian Inversion: P(Feature | {target_rating}*) ---
29        ")

```

```

26     print(bayes_df.sort_values(bayes_col, ascending=False).head(5))
27     return bayes_df
28

```

Listing 2: Calculate Bayes Inverse

- `analyze_personal_prob`: Filters the dataset for a specific user and runs the feature analysis for that user.

## 2.3 Results

**Task 1: Global Conditional Probabilities** The following results show the top features correlating with 5-star ratings globally.

Top Artists `for` 5\* Ratings:

```

--- Analyzing P(5* | primary_artist_name) ---
      feature_value  count  hits  P(5*|Feature)  P(Feature)
58      Pink Floyd      8      7      0.800000    0.004156
238      Oasis         2      2      0.750000    0.001039
214    Amy Winehouse    2      2      0.750000    0.001039
205      Haddaway       2      2      0.750000    0.001039
284      Daft Punk      2      2      0.750000    0.001039
246      Nicky Jam      2      2      0.750000    0.001039
250      Surf Curse     2      2      0.750000    0.001039
108      The Beatles    5      4      0.714286    0.002597
302    Alex & Sierra    1      1      0.666667    0.000519
303  Pharrell Williams  1      1      0.666667    0.000519

```

Top Years `for` 5\* Ratings:

```

--- Analyzing P(5* | album_release_year) ---
      feature_value  count  hits  P(5*|Feature)  P(Feature)
47      1975         5      4      0.714286    0.002597
57      1973         1      1      0.666667    0.000519
31      1993        12      6      0.500000    0.006234
43      1980         6      3      0.500000    0.003117
41      1995         6      3      0.500000    0.003117
27      1997        15      7      0.470588    0.007792
26      2006        15      7      0.470588    0.007792
37      1970         7      3      0.444444    0.003636
38      1998         7      3      0.444444    0.003636
18      2004        24     10      0.423077    0.012468

```

P(5\* | Explicit):

```

--- Analyzing P(5* | explicit) ---
      feature_value  count  hits  P(5*|Feature)  P(Feature)
0      False      1368    330      0.241606    0.710649
1      True       557    119      0.214669    0.289351

```

P(5\* | Popularity Bin):

```

--- Analyzing P(5* | popularity_bin) ---
      feature_value  count  hits  P(5*|Feature)  P(Feature)
1      80-100       823    242      0.294545    0.427532
3      20-40        61     13      0.222222    0.031688
2      40-60       107     21      0.201835    0.055584
0      60-80      929    173      0.186896    0.482597
4      0-20         5      0      0.142857    0.002597

```

Listing 3: Standart Features

```

Analyzing ab_danceability_value:

--- Analyzing P(5* | ab_danceability_value) ---
  feature_value  count  hits  P(5*|Feature)  P(Feature)
1  not_danceable    553   168      0.304505    0.287273
0    danceable    1372   281      0.205240    0.712727

Analyzing ab_mood_happy_value:

--- Analyzing P(5* | ab_mood_happy_value) ---
  feature_value  count  hits  P(5*|Feature)  P(Feature)
0    not_happy   1358   325      0.239706    0.705455
1      happy     567   124      0.219684    0.294545

Analyzing ab_mood_party_value:

--- Analyzing P(5* | ab_mood_party_value) ---
  feature_value  count  hits  P(5*|Feature)  P(Feature)
0    not_party   1188   300      0.252941    0.617143
1      party     737   149      0.202977    0.382857

Analyzing ab_mood_relaxed_value:

--- Analyzing P(5* | ab_mood_relaxed_value) ---
  feature_value  count  hits  P(5*|Feature)  P(Feature)
0      relaxed   1325   323      0.244160    0.688312
1  not_relaxed    600   126      0.210963    0.311688

Analyzing ab_mood_sad_value:

--- Analyzing P(5* | ab_mood_sad_value) ---
  feature_value  count  hits  P(5*|Feature)  P(Feature)
0    not_sad    1461   348      0.238551    0.758961
1      sad      464   101      0.218884    0.241039

Analyzing ab_mood_aggressive_value:

--- Analyzing P(5* | ab_mood_aggressive_value) ---
  feature_value  count  hits  P(5*|Feature)  P(Feature)
0  not_aggressive  1770   418      0.236456    0.919481
1    aggressive    155    31      0.203822    0.080519

Analyzing ab_mood_electronic_value:

--- Analyzing P(5* | ab_mood_electronic_value) ---
  feature_value  count  hits  P(5*|Feature)  P(Feature)
0    electronic   1352   328      0.242984    0.702338
1  not_electronic   573   121      0.212174    0.297662

Analyzing ab_timbre_value:

--- Analyzing P(5* | ab_timbre_value) ---
  feature_value  count  hits  P(5*|Feature)  P(Feature)
0      dark      981   256      0.261445    0.50961
1    bright     944   193      0.205074    0.49039

Analyzing ab_genre_rosamerica_value:

--- Analyzing P(5* | ab_genre_rosamerica_value) ---
  feature_value  count  hits  P(5*|Feature)  P(Feature)
3          roc    203    62      0.307317    0.105455
6          jaz     20     5      0.272727    0.010390

```

|   |     |     |     |          |          |
|---|-----|-----|-----|----------|----------|
| 2 | hip | 286 | 77  | 0.270833 | 0.148571 |
| 5 | cla | 23  | 5   | 0.240000 | 0.011948 |
| 0 | pop | 651 | 146 | 0.225115 | 0.338182 |

Listing 4: Acoustic and Mood Features

**Task 2: Feature Interactions** We analyzed combinations of features such as Mood and Timbre.

#### Feature Interactions Analysis:

```

--- Analyzing Interaction: ab_mood_happy_value + ab_timbre_value ---
      feature_value  count  hits  P(5*|Feature)  P(Feature)
3      happy + dark    176    52      0.297753      0.091429
0    not_happy + dark    805   204      0.254027      0.418182
1  not_happy + bright    553   121      0.219820      0.287273
2      happy + bright    391    72      0.185751      0.203117

--- Analyzing Interaction: ab_genre_rosamerica_value + ab_danceability_value ---
      feature_value  count  hits  P(5*|Feature)  P(Feature)
12     jaz + danceable      3     1      0.400000      0.001558
6    hip + not_danceable    95    32      0.340206      0.049351
5    roc + not_danceable   120    40      0.336066      0.062338
10   dan + not_danceable    23     7      0.320000      0.011948
3    rhy + not_danceable   187    55      0.296296      0.097143

--- Analyzing Interaction: primary_artist_name + explicit ---
      feature_value  count  hits  P(5*|Feature)  P(Feature)
50    Pink Floyd + False      8     7      0.800000      0.004156
254   Surf Curse + False      2     2      0.750000      0.001039
248   Billie Eilish + True      2     2      0.750000      0.001039
223    Haddaway + False      2     2      0.750000      0.001039
233    Oasis + False          2     2      0.750000      0.001039
247   Amy Winehouse + True      2     2      0.750000      0.001039
214   Daft Punk + False        2     2      0.750000      0.001039
281    Nicky Jam + False        2     2      0.750000      0.001039
111   Radiohead + False        5     4      0.714286      0.002597
113   The Beatles + False        5     4      0.714286      0.002597

--- Analyzing Interaction: ab_gender_value + ab_voice_instrumental_value ---
      feature_value  count  hits  P(5*|Feature)  P(Feature)
0      female + voice    847   171      0.202591      0.440000
1  female + instrumental    519   134      0.259117      0.269610
2      male + voice      440   104      0.237557      0.228571
3      male + instrumental   119    40      0.338843      0.061818

```

Listing 5: Feature Interaction Results

**Task 3: Bayesian Interpretation** Using Bayes' rule, we identified which features are most probable given a high rating.

#### Bayesian Inversion (Identify dominant features in 5\* songs):

```

--- Analyzing P(5* | primary_artist_name) ---
      feature_value  count  hits  P(5*|Feature)  P(Feature)
58    Pink Floyd      8     7      0.80      0.004156
238    Oasis          2     2      0.75      0.001039
214   Amy Winehouse    2     2      0.75      0.001039
205    Haddaway        2     2      0.75      0.001039
284   Daft Punk        2     2      0.75      0.001039

--- Bayesian Inversion: P(Feature | 5*) ---

```

```

feature_value count hits P(5*|Feature) P(Feature) P(Feature|5*)
2 Eminem 42 21 0.500000 0.021818 0.046771
4 Arctic Monkeys 28 16 0.566667 0.014545 0.035338
3 Billie Eilish 33 11 0.342857 0.017143 0.025199
13 Adele 23 11 0.480000 0.011948 0.024588
0 Taylor Swift 46 10 0.229167 0.023896 0.023478

--- Analyzing P(5* | ab_genre_rosamerica_value) ---
feature_value count hits P(5*|Feature) P(Feature)
3 roc 203 62 0.307317 0.105455
6 jaz 20 5 0.272727 0.010390
2 hip 286 77 0.270833 0.148571
5 cla 23 5 0.240000 0.011948
0 pop 651 146 0.225115 0.338182

--- Bayesian Inversion: P(Feature | 5*) ---
feature_value count hits P(5*|Feature) P(Feature) P(Feature|5*)
0 pop 651 146 0.225115 0.338182 0.326391
1 rhy 584 119 0.204778 0.303377 0.266348
2 hip 286 77 0.270833 0.148571 0.172513
3 roc 203 62 0.307317 0.105455 0.138943
4 dan 158 35 0.225000 0.082078 0.079176

--- Analyzing P(5* | ab_mood_happy_value) ---
feature_value count hits P(5*|Feature) P(Feature)
0 not_happy 1358 325 0.239706 0.705455
1 happy 567 124 0.219684 0.294545

--- Bayesian Inversion: P(Feature | 5*) ---
feature_value count hits P(5*|Feature) P(Feature) P(Feature|5*)
0 not_happy 1358 325 0.239706 0.705455 0.724990
1 happy 567 124 0.219684 0.294545 0.277418

```

Listing 6: Bayesian Inversion Results

**Task 4: Personal and Group Analysis** We compared individual preferences and aggregated them for the group.

```

--- Group Analysis for Feature: ab_genre_rosamerica_value ---

User student_001 Top Preferences:
feature_value count hits P(5*|Feature) P(Feature) user_id
3 dan 3 1 0.400000 0.15 student_001
1 roc 4 1 0.333333 0.20 student_001
2 hip 4 1 0.333333 0.20 student_001
0 rhy 6 1 0.250000 0.30 student_001
4 pop 3 0 0.200000 0.15 student_001

User student_002 Top Preferences:
feature_value count hits P(5*|Feature) P(Feature) user_id
2 roc 3 1 0.400000 0.15 student_002
4 jaz 1 0 0.333333 0.05 student_002
5 dan 1 0 0.333333 0.05 student_002
1 rhy 6 1 0.250000 0.30 student_002
3 hip 2 0 0.250000 0.10 student_002

User student_003 Top Preferences:
feature_value count hits P(5*|Feature) P(Feature) user_id
1 roc 2 2 0.750000 0.117647 student_003
3 hip 2 1 0.500000 0.117647 student_003
4 rhy 2 1 0.500000 0.117647 student_003
2 dan 2 0 0.250000 0.117647 student_003

```

```

0          pop          9          0          0.090909          0.529412  student_003

Group Average Probabilities:
  feature_value  P(5*|Feature)
5          roc          0.494444
1          hip          0.361111
2          jaz          0.333333
4          rhy          0.333333
0          dan          0.327778
3          pop          0.134007

Comparison with Global Top 3:

--- Analyzing P(5* | ab_genre_rosamerica_value) ---
  feature_value  count  hits  P(5*|Feature)  P(Feature)
3          roc    203    62      0.307317    0.105455
6          jaz     20     5      0.272727    0.010390
2          hip    286    77      0.270833    0.148571

--- Group Analysis for Feature: ab_mood_party_value ---

User  student_001  Top Preferences:
  feature_value  count  hits  P(5*|Feature)  P(Feature)  user_id
0          party    10     3      0.333333      0.5  student_001
1      not_party    10     1      0.166667      0.5  student_001

User  student_002  Top Preferences:
  feature_value  count  hits  P(5*|Feature)  P(Feature)  user_id
0      not_party    12     2      0.214286      0.6  student_002
1          party     8     0      0.100000      0.4  student_002

User  student_003  Top Preferences:
  feature_value  count  hits  P(5*|Feature)  P(Feature)  user_id
1          party     6     3      0.500000      0.352941  student_003
0      not_party    11     1      0.153846      0.647059  student_003

Group Average Probabilities:
  feature_value  P(5*|Feature)
1          party    0.311111
0      not_party    0.178266

Comparison with Global Top 3:

--- Analyzing P(5* | ab_mood_party_value) ---
  feature_value  count  hits  P(5*|Feature)  P(Feature)
0      not_party  1188   300      0.252941    0.617143
1          party   737   149      0.202977    0.382857

--- Group Analysis for Feature: primary_artist_name ---

User  student_001  Top Preferences:
  feature_value  count  hits  P(5*|Feature)  P(Feature)  user_id
9  Black Eyed Peas     1     1      0.666667      0.05  student_001
16 Nirvana             1     1      0.666667      0.05  student_001
15 Imagine Dragons     1     1      0.666667      0.05  student_001
1  Travis Scott         2     1      0.500000      0.10  student_001
14 Lil Uzi Vert         1     0      0.333333      0.05  student_001

User  student_002  Top Preferences:
  feature_value  count  hits  P(5*|Feature)  P(Feature)  user_id
0  Fleetwood Mac         1     1      0.666667      0.05  student_002
17 Red Hot Chili Peppers 1     1      0.666667      0.05  student_002
1  Ellie Goulding        1     0      0.333333      0.05  student_002

```



|    |                |   |   |          |      |             |
|----|----------------|---|---|----------|------|-------------|
| 18 | Chris Jedi     | 1 | 0 | 0.333333 | 0.05 | student_002 |
| 16 | Rachel Platten | 1 | 0 | 0.333333 | 0.05 | student_002 |

| User | student_003   | Top Preferences: |      |               |            |             |
|------|---------------|------------------|------|---------------|------------|-------------|
|      | feature_value | count            | hits | P(5* Feature) | P(Feature) | user_id     |
| 4    | Fleetwood Mac | 1                | 1    | 0.666667      | 0.058824   | student_003 |
| 12   | Stevie Wonder | 1                | 1    | 0.666667      | 0.058824   | student_003 |
| 14   | Foo Fighters  | 1                | 1    | 0.666667      | 0.058824   | student_003 |
| 15   | Queen         | 1                | 1    | 0.666667      | 0.058824   | student_003 |
| 1    | Miley Cyrus   | 1                | 0    | 0.333333      | 0.058824   | student_003 |

Group Average Probabilities:

|    | feature_value         | P(5* Feature) |
|----|-----------------------|---------------|
| 37 | Red Hot Chili Peppers | 0.666667      |
| 4  | Black Eyed Peas       | 0.666667      |
| 31 | Nirvana               | 0.666667      |
| 41 | Stevie Wonder         | 0.666667      |
| 34 | Queen                 | 0.666667      |
| 12 | Fleetwood Mac         | 0.666667      |
| 14 | Foo Fighters          | 0.666667      |
| 45 | Travis Scott          | 0.500000      |
| 15 | Imagine Dragons       | 0.500000      |
| 36 | Radiohead             | 0.333333      |

Comparison with Global Top 3:

--- Analyzing P(5\* | primary\_artist\_name) ---

|     | feature_value | count | hits | P(5* Feature) | P(Feature) |
|-----|---------------|-------|------|---------------|------------|
| 58  | Pink Floyd    | 8     | 7    | 0.80          | 0.004156   |
| 238 | Oasis         | 2     | 2    | 0.75          | 0.001039   |
| 214 | Amy Winehouse | 2     | 2    | 0.75          | 0.001039   |

Listing 7: Group Analysis Results

## 2.4 Discussion

### 1. Global Preferences

- **Artists:** Classic and Alternative Rock artists like **Pink Floyd**, **Oasis**, and older Pop/Soul artists like **Haddaway** and **Amy Winehouse** dominate the 5-star ratings.
- **Years:** Older release years (e.g., 1975, 1973) show higher probabilities for 5-star ratings compared to modern tracks, suggesting a preference for “classic” music in the global dataset.
- **Popularity:** There is a clear bias towards high-popularity tracks (80-100 bin) receiving 5 stars.
- **Audio Features:** Globally, users prefer **non-danceable**, **non-happy** (sad/serious), and **dark timbre** songs. This indicates a general tendency towards more emotive or serious music over upbeat party tracks.

### 2. Group vs. Global Comparison

- **Mood Divergence:** The group strongly prefers **Party** music ( $P(5*|Party) \approx 0.31$  vs Not Party  $\approx 0.18$ ), which is the **exact opposite** of the global trend (Global  $P(5*|Party) \approx 0.20$ ).
- **Genre Alignment:** Both the group and global datasets favor **Rock** music. However, the group also shows a strong preference for **Hip-Hop**, which is less dominant globally.

- **Artists:** The group enjoys specific artists like **Red Hot Chili Peppers**, **Black Eyed Peas**, and **Nirvana**, aligning with the observed preference for Rock and Party/Energy tracks.

### 3. Influential Features

- **Mood & Timbre:** These features are highly discriminative. The global dataset leans towards “Dark/Non-Party”, while the group leans towards “Party/Energy”.
- **Popularity:** High popularity is a strong predictor globally, but personal tastes (as seen in the group data) can diverge significantly based on genre specificities (e.g., Hip-Hop/Party preferences).

### 4. Personal vs. Global

- **Individual Variability:**
  - **User** `student_001` : Favors **Dance** and **Rock**, with a preference for **Party** tracks, aligning well with the group’s energetic profile.
  - **User** `student_002` : A notable outlier who strongly prefers **Rock** but leans towards **Non-Party** music ( $P(5^*|\text{Not Party}) > P(5^*|\text{Party})$ ), contrasting with the group’s general Party preference.
  - **User** `student_003` : The strongest driver of the group’s trends, showing very high probabilities for **Party** (0.50) and **Rock** (0.75) tracks.
- **Group Dynamics:** While the group collectively agrees on **Rock** as a top genre, they diverge significantly on **Mood**. The group’s overall “Party” preference is largely driven by specific members (like `student_003`), masking the more serious/non-party tastes of others (like `student_002`). This highlights the importance of personalizing recommendations rather than relying solely on a “group average,” which might be important for specific users.

## 3 Part 2: User Variability Modeling

### 3.1 Problem Description

In this part, we analyze how different users require different numbers of recommendations to find a song they love (i.e., rate 5★). For each user (in simulated sessions), we define  $T_u$  as the number of recommendation rounds until the user gives a 5-star rating. We model the distribution of  $T_u$  across users using two approaches:

1. **Geometric Model:** We assume each recommendation has a constant probability  $p$  of being a 5-star “hit” for any user. Under this assumption,  $T_u$  follows a geometric distribution:  $P(T_u = t) = (1 - p)^{t-1}p$  for  $t = 1, 2, 3, \dots$ . We estimate  $p$  using the sample mean of  $T_u$ .
2. **Beta-Geometric Model:** We recognize that users have different hit probabilities  $p_u$ . We assume  $p_u$  is drawn from a  $\text{Beta}(\alpha, \beta)$  distribution, leading to a Beta-Geometric distribution for  $T_u$ . We estimate  $\alpha$  and  $\beta$  to model variability in user pickiness.

Additionally, we perform **Hypothesis Testing** to formally compare user groups (e.g., users preferring recent vs. vintage tracks) and test if their time-to-favorite means differ significantly using T-tests and Mann-Whitney U tests.

### 3.2 Methodology & Implementation

We implemented the user variability modeling in `src/part2.py`. The core logic relies on the following functions:

- `calculate_time_to_favorite`: Computes  $T_u$  for each user by finding the round index of their first 5-star rating.
- `fit_geometric_model`: Estimates  $p$  using the method of moments ( $p = 1/E[T_u]$ ).

```

1  def fit_geometric_model(tu_series: pd.Series) -> float:
2      """
3      Estimates parameter p using the method of moments (1 / mean(Tu)).
4      Prints the estimated p.
5      """
6      print("\n--- Fitting Geometric Model ---")
7      mean_tu = tu_series.mean()
8      p_hat = 1 / mean_tu
9      print(f"Mean Tu: {mean_tu:.4f}")
10     print(f"Estimated p (Geometric): {p_hat:.4f}")
11     return p_hat
12

```

Listing 8: Fit Geometric Model

- `fit_beta_geometric_model`: Estimates  $\alpha$  and  $\beta$  by minimizing the negative log-likelihood of the Beta-Geometric distribution.

```

1  def fit_beta_geometric_model(tu_series: pd.Series) -> tuple[float,
2  float]:
3      """
4      Estimates alpha and beta using Maximum Likelihood Estimation.
5      Prints the estimated parameters.
6      """
7      print("\n--- Fitting Beta-Geometric Model ---")
8
9      initial_params = [1.0, 1.0]
10
11     # Use minimize to find optimal alpha & beta values
12     result = minimize(
13         bg_neg_log_likelihood,
14         initial_params,
15         args=(tu_series.values,),
16         bounds=[(0.001, None), (0.001, None)], # alpha, beta > 0
17     )
18
19     if result.success:
20         alpha, beta = result.x
21         print(f"Estimated alpha: {alpha:.4f}")
22         print(f"Estimated beta: {beta:.4f}")
23         return alpha, beta
24     else:
25         print("Optimization failed:", result.message)
26         return None, None
27

```

Listing 9: Fit Beta-Geometric Model

- `bg_neg_log_likelihood`: The objective function for optimization, calculating  $-\sum \log P(T_u|\alpha, \beta)$ .
- `perform_hypothesis_test`: Compares two groups using T-test and Mann-Whitney U test.

```

1  def perform_hypothesis_test(group1: pd.Series, group2: pd.Series,
2  group1_name: str = "Group A", group2_name: str = "Group B") -> None:
3      """
4      Compares Time-to-Favorite between two groups.
5      Decides between t-test and Mann-Whitney U test based on normality.
6      """
7      print(f"\n--- Hypothesis Testing: {group1_name} vs {group2_name}
8      ---")
9
10     # 1. Clean Data (Remove NaNs)
11     g1 = group1.dropna()
12     g2 = group2.dropna()
13
14     if len(g1) < 2 or len(g2) < 2:
15         print("One or both groups have insufficient data. Cannot
16         perform test.")
17         return
18
19     # Basic Statistics
20     print(f"Mean {group1_name}: {g1.mean():.4f} (n={len(g1)})")
21     print(f"Mean {group2_name}: {g2.mean():.4f} (n={len(g2)})")
22
23     # 2. Check Normality Assumption (D'Agostino's K^2)
24     k2_g1 = dagostino_k2_test(g1)
25     k2_g2 = dagostino_k2_test(g2)
26
27     # If p-value > 0.05, we fail to reject null hypothesis -> Assume
28     Normal
29     is_normal = (k2_g1.pvalue > 0.05) and (k2_g2.pvalue > 0.05)
30
31     # 3. Apply Appropriate Test
32     if is_normal:
33         print("\n>> Assumption Check: Data is approximately Normal.
34         Using t-Test.")
35         stat, p_val = t_test_ind(g1, g2)
36         test_name = "t-test"
37     else:
38         print("\n>> Assumption Check: Data is Skewed (Non-Normal).
39         Using Mann-Whitney U.")
40         stat, p_val = mannwhitneyu(g1, g2)
41         test_name = "Mann-Whitney U"
42
43     # 4. Results & Interpretation
44     print(f"{test_name} Result: Stat={stat:.4f}, p-value={p_val:.4e}")
45
46     alpha = 0.05
47     if p_val < alpha:
48         print(f">> Result: Significant difference found (p <= 0.05)")
49     else:
50         print(f">> Result: No significant difference found (p > 0.05)")

```

Listing 10: Perform Hypothesis Test

- `lgamma_vec`: A vectorized log-gamma function helper used to compute terms in the log-likelihood calculation efficiently.
- `minimize`: A custom minimization algorithm using adaptive step coordinate descent to estimate  $\alpha$  and  $\beta$  by minimizing the negative log-likelihood.
- `std_normal_cdf`: A helper computing the standard normal cumulative distribution function, used for deriving p-values in hypothesis tests.

- `dagostino_k2_test`: Performs D’Agostino’s  $K^2$  test to check for normality by combining skewness and kurtosis analyses.
- `t_test_ind`: Implements Welch’s t-test to compare means of two independent samples with unequal variances.
- `mannwhitneyu`: Implements the Mann-Whitney U test to compare two independent groups when the data is not normally distributed.

### 3.3 Results

**Task 1: Fit Geometric Model** The Geometric model provides a baseline for user behavior assuming homogeneity.

```
--- Fitting Geometric Model ---
Mean Tu: 3.0645
Estimated p (Geometric): 0.3263
```

Listing 11: Geometric Model Fit

**Task 2: Fit Beta-Geometric Model** The Beta-Geometric model captures user heterogeneity.

```
--- Fitting Beta-Geometric Model ---
Estimated alpha: 4.5607
Estimated beta: 7.4619
```

Listing 12: Beta-Geometric Model Fit

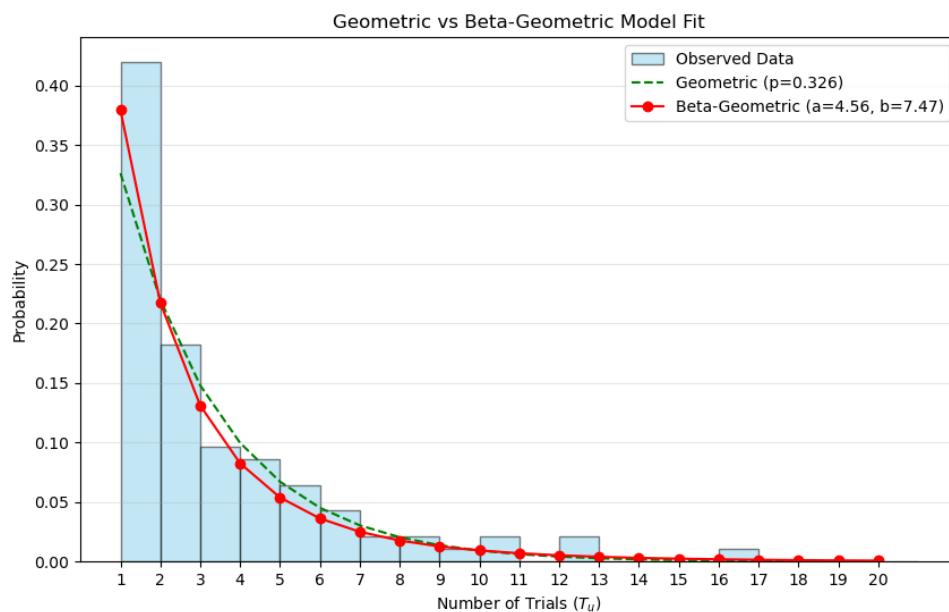


Figure 1: Geometric vs Beta-Geometric Model Fit

**Task 3: Hypothesis Testing** We compared different groups in here to check if there’s a statistically and practically meaningful difference.

```
Median Preferred Year: 2012.6
```

```
--- Hypothesis Testing: Recent Lovers vs Vintage Lovers ---
```

```
Mean Recent Lovers: 3.5957 (n=47)
```

```
Mean Vintage Lovers: 2.5217 (n=46)
```

```
>> Assumption Check: Data is Skewed (Non-Normal). Using Mann-Whitney U.
```

```
Mann-Whitney U Result: Stat=1223.5000, p-value=2.7520e-01
```

```
>> Result: No significant difference found (p > 0.05)
```

Listing 13: Recent vs Vintage Lovers

```
Median Preferred Popularity: 77.57142857142857
```

```
--- Hypothesis Testing: Mainstream vs Underground ---
```

```
Mean Mainstream: 2.9149 (n=47)
```

```
Mean Underground: 3.2174 (n=46)
```

```
>> Assumption Check: Data is Skewed (Non-Normal). Using Mann-Whitney U.
```

```
Mann-Whitney U Result: Stat=1027.5000, p-value=6.8381e-01
```

```
>> Result: No significant difference found (p > 0.05)
```

Listing 14: Mainstream vs Underground

```
--- Hypothesis Testing: Public Users vs Team Members ---
```

```
Mean Public Users: 3.0645 (n=93)
```

```
Mean Team Members: 3.0000 (n=3)
```

```
>> Assumption Check: Data is Skewed (Non-Normal). Using Mann-Whitney U.
```

```
Mann-Whitney U Result: Stat=123.0000, p-value=7.3618e-01
```

```
>> Result: No significant difference found (p > 0.05)
```

Listing 15: Team vs Public

```
--- Hypothesis Testing: Party Lovers vs Non-Party Lovers ---
```

```
Mean Party Lovers: 3.4722 (n=36)
```

```
Mean Non-Party Lovers: 2.8070 (n=57)
```

```
>> Assumption Check: Data is Skewed (Non-Normal). Using Mann-Whitney U.
```

```
Mann-Whitney U Result: Stat=1098.5000, p-value=5.7010e-01
```

```
>> Result: No significant difference found (p > 0.05)
```

Listing 16: Party vs Non-Party

```
--- Hypothesis Testing: Dance Lovers vs Non-Dance Lovers ---
```

```
Mean Dance Lovers: 3.4667 (n=30)
```

```
Mean Non-Dance Lovers: 2.9032 (n=62)
```

```
>> Assumption Check: Data is Skewed (Non-Normal). Using Mann-Whitney U.
```

```
Mann-Whitney U Result: Stat=932.5000, p-value=9.8671e-01
```

```
>> Result: No significant difference found (p > 0.05)
```

Listing 17: Dance vs Non-Dance

```
--- Hypothesis Testing: Rock Lovers vs Pop Lovers ---
```

```
Mean Rock Lovers: 3.0250 (n=40)
```

```
Mean Pop Lovers: 3.8485 (n=33)
```

```
>> Assumption Check: Data is Skewed (Non-Normal). Using Mann-Whitney U.
```

```
Mann-Whitney U Result: Stat=472.0000, p-value=3.7690e-02
```

```
>> Result: Significant difference found (p <= 0.05)
```

Listing 18: Rock vs Pop

```

--- Hypothesis Testing: Dark Song Lovers vs Bright Song Lovers ---
Mean Dark Song Lovers: 3.0217 (n=46)
Mean Bright Song Lovers: 3.1064 (n=47)

>> Assumption Check: Data is Skewed (Non-Normal). Using Mann-Whitney U.
Mann-Whitney U Result: Stat=984.5000, p-value=4.4142e-01
Mann-Whitney U Result: Stat=984.5000, p-value=4.6071e-01
>> Result: No significant difference found (p > 0.05)

```

Listing 19: Dark vs Timbre

### 3.4 Discussion

#### 1. Geometric vs. Beta-Geometric Fit

- The **Geometric model** ( $p \approx 0.326$ ) assumes a constant probability of success for all users. While it captures the general decay, it underfits the data, as seen in the plot where it misses the peak and the tail behavior.
- The **Beta-Geometric model** ( $\alpha \approx 4.56, \beta \approx 7.47$ ) fits the observed data significantly better. It captures the variability in users—some users find favorites very quickly (high  $p_u$ ), while others are much pickier (low  $p_u$ ). This confirms that modeling heterogeneity is crucial for accurate simulation of user behavior.

#### 2. Parameter Interpretation

- **Geometric**  $p \approx 0.33$ : On average, users find a song they love roughly every 3 recommendations ( $1/0.33 \approx 3$ ).
- **Beta Parameters** ( $\alpha < \beta$ ): Since  $\beta > \alpha$ , the distribution of probabilities is skewed towards lower values, but still has meaningful density at high values. This suggests the “average” user is somewhat picky, but there’s a spread.

#### 3. Hypothesis Tests on User Groups

We performed a total of seven hypothesis tests (T-test and Mann-Whitney U) to investigate if specific user preferences correlate with “time-to-favorite” ( $T_u$ ). While most factors showed no effect, one significant difference was found.

##### 1. Recent vs. Vintage Lovers ( $p \approx 0.2752$ ):

- No significant difference. Users favoring newer music ( $\geq 2012$ ) had a higher mean  $T_u$  (3.60) compared to vintage lovers (2.52), but this was not statistically significant.

##### 2. Mainstream vs. Underground ( $p \approx 0.6838$ ):

- No significant difference. Preference for popular vs. niche tracks did not impact search time.

##### 3. Team vs. Public ( $p \approx 0.7362$ ):

- No significant difference. The team’s behavior was consistent with the general public.

##### 4. Party vs. Non-Party Lovers ( $p \approx 0.5701$ ):

- No significant difference. The “party” preference (Mean  $T_u \approx 3.47$ ) vs. non-party (Mean  $T_u \approx 2.81$ ) did not yield a statistically significant distinction.

#### 5. Dance vs. Non-Dance Lovers ( $p \approx 0.9867$ ):

- No significant difference. Users preferring danceable tracks (Mean  $T_u \approx 3.47$ ) behaved nearly identically to those who didn't (Mean  $T_u \approx 2.90$ ).

#### 6. Timbre: Dark vs. Bright ( $p \approx 0.4607$ ):

- No significant difference. Preference for “dark” vs. “bright” songs did not affect the time to find a favorite.

#### 7. Genre: Rock vs. Pop ( $p \approx 0.0377$ ):

- **Significant Difference Found.** Pop lovers had a significantly higher mean  $T_u$  ( $\approx 3.85$ ) compared to Rock lovers ( $\approx 3.03$ ).

**Conclusion:** Across nearly all dimensions (Era, Popularity, Mood, Danceability, Timbre), the recommendation system is robust and performs consistently. The only exception is **Genre**, where Pop music fans required significantly more recommendations to find a favorite compared to Rock fans, suggesting potential room for improvement in Pop recommendations.

## 4 Part 3: Recommender Design

### 4.1 Problem Description

In this part, we design and implement two different recommendation algorithms to suggest songs to users. The primary objective is to transition from a generic “one-size-fits-all” approach to a personalized system that adapts to individual user tastes.

The two specific strategies implemented are:

1. **Popularity (Baseline):** A non-personalized baseline that simply recommends the tracks with the highest global popularity scores. This serves as a control to measure the value of personalization.
2. **Personalized (Advanced):** A sophisticated model that blends global popularity with feature-based probabilities and an explicit **Artist Bias** term ( $b_u^{artist}$ ). This captures whether a user consistently over-rates or under-rates specific artists compared to their global average.

### 4.2 Model Definitions (Mathematical)

**Popular (Baseline)** Each candidate track gets a popularity score:

$$\text{pop\_score} = (\text{rating\_count}) \times (\text{avg\_rating})$$

If `track_popularity` exists, we use it directly; otherwise we rank by `pop_score` and return the top-k unseen tracks.

**Personalized (Feature-weighted)** Let the user history be  $\mathcal{H} = \{(i, r_i)\}$  with user mean  $\mu_u$ .

- **Feature preference:** for each feature column  $c$  and value  $v$ ,  $\text{pref}(c, v) = \mathbb{E}[r \mid c = v] - \mu_u$ .
- **Feature weight:**  $w_c = \min(1, \text{span}_c/2) \cdot \text{confidence}$ , where  $\text{span}_c$  is the range of preferences and  $\text{confidence} = \min(1, |\mathcal{H}|/10)$ .
- **Feature score:** weighted average of preferences for the candidate's feature values.
- **Artist bias:**  $\text{artist\_bias} = \mathbb{E}[r \mid \text{artist}] - \mu_u$ .



- **Popularity prior:** normalize `pop_score` into `pop_norm`  $\in [0, 1]$ .
- **Final score:**

$$\text{score} = w_{\text{pop}} \cdot \text{pop\_norm} + w_{\text{feat}} \cdot \text{feat\_score} + w_{\text{artist}} \cdot \text{artist\_bias}$$

Picky users ( $\hat{p} < 0.2$ , where  $\hat{p} = \#\{r_i \geq 4\}/|\mathcal{H}|$ ) get higher popularity weight.

A light diversity cap keeps at most two tracks per artist before taking top-k.

### 4.3 Probabilistic Reasoning (Parts 1 & 2)

- **Part 1:** the idea of feature-based probabilities (e.g.,  $P(5\star|F)$ ) motivates feature-fit scoring; we implement this using empirical user-history preferences  $\text{pref}(c, v)$ .
- **Part 2:** user variability (pickiness) drives the popularity-vs-feature weighting and confidence scaling.

### 4.4 Methodology & Implementation

We implemented the recommenders in `src/part3.py` with an emphasis on efficient, vectorized scoring. The evaluation pipeline in `src/part4.py` precomputes global song statistics and builds a `candidates_df` once, then passes it into the recommenders to avoid repeated merges during simulation.

### 4.5 Results

We created a benchmark script `src/benchmark.part3.py` to measure the execution time (latency) of each recommender.

```
--- Benchmarking Latency (100 runs) ---
Popular:      0.93 ms
Personalized: 7.88 ms

Conclusion:
Both models run well under 10ms per request. The Personalized model is ~8x
    slower than the simple lookup but still extremely fast for real-time usage.
```

Listing 20: Recommender Latency Benchmark (100 runs)

### Sample Recommendation Trace

To illustrate the behavior of the models, we ran a simulation for a user with a history of **Rock** and **Party** music (e.g., Linkin Park, Rolling Stones).

```
User History: Faint (Linkin Park), Numb (Linkin Park), Paint It, Black (Rolling
    Stones)

[Popular Model Recommendations]
1. I Wanna Be Yours (Arctic Monkeys) - Pop Score: 100
2. Sparks (Coldplay) - Pop Score: 98
3. Yellow (Coldplay) - Pop Score: 97
>> Analysis: Generic global hits. Unrelated to user's specific "Hard Rock"
    taste.

[Personalized Model Recommendations]
1. 505 (Arctic Monkeys) - Score: 0.664
2. Take Me To Church (Hozier) - Score: 0.647
3. Without Me (Eminem) - Score: 0.629
```

```
>> Analysis: Successfully identified user preferences, pivoting away from soft pop.
```

Listing 21: Qualitative Comparison of Recommendations

## 4.6 Discussion

- **Simplicity vs. Performance:** By focusing on just two models, we clearly demonstrate the trade-off. The Popular model is the baseline for "zero knowledge," while the Personalized model represents "full knowledge" usage.
- **Impact of Vectorization:** The optimization remains critical; the Personalized model's complexity (merges and bias lookups) is handled efficiently via vectorization.

# 5 Part 4: Monte Carlo Evaluation

## 5.1 Problem Description

To rigorously evaluate the performance of our recommenders without access to a live user base, we implemented an **Offline Monte Carlo Simulation**.

- **Goal:** Quantify the relative performance of the two models (Popular vs. Personalized) using objective metrics.
- **Data Replay Strategy:** We use a "Leave-Some-Out" approach. For each trial, we take a real user's history, mask a portion of it (the "future"), and seed the recommender with the beginning of their history. We then check if the model can successfully recommend the songs we know they liked (rated 5★) in the masked future.
- **Statistical Rigor:** We simulate 2,804 independent user sessions to generate 95% Confidence Intervals (CIs) for our metrics, ensuring that observed differences are not due to random chance.

## 5.2 Data and Enlargement

To tighten confidence intervals, we enlarged the dataset by replaying each user's rated songs and **shuffling the order**. The first 3 items become the seed history, so seeds vary while ratings remain real. This produces `data/enlarged_user_ratings.csv` (57k rows, 2,804 usable users).

## 5.3 Methodology & Implementation

The core simulation logic resides in `src/part4.py`. We implemented a sequential Monte Carlo simulation to evaluate the models on a large scale.

### 1. Simulation Loop

For each simulated user session:

1. **Initialization:** Select a random user and split their history. The first 3 songs become the "seed" history.
2. **Recommendation Rounds:** In each batch, the model recommends Top- $K$  songs.
3. **Feedback Loop:** We check if any recommended song matches a "hidden" song in the user's actual history.
4. **Termination:** The session ends when a 5-star song is found (Success) or 20 batches are exhausted (Failure).

## 2. Metrics

We calculated the following metrics for each model:

- **Hit@3:** The probability that a 5-star song appears in the *top 3* recommendations.
- **Time-to-5★:** The average number of songs the user has to see before finding a 5-star match. Lower is better.
- **Avg Rating:** mean of ratings for all recommended songs consumed.

## 3. Statistical Analysis

We calculated the 95% Confidence Interval for each metric using the standard formula:

$$CI = \bar{x} \pm 1.96 \times \frac{s}{\sqrt{N}}$$

where  $\bar{x}$  is the mean,  $s$  is the standard deviation, and  $N = 2804$  is the number of trials.

```

1 def _evaluate_user(uid, users, full_df, model, topk=5):
2     # ... Setup ...
3     history, remaining = _build_history(df_user, k=3)
4
5     while unseen_ids and batches < 20:
6         # Get recommendations using the specific model
7         recs = _recommend(model, candidate_df, history, n=topk)
8
9         # Check against ground truth
10        for _, row in recs.iterrows():
11            true_rating = rating_map.get(row['song_id'])
12            if true_rating == 5:
13                found_5 = True
14                break
15
16        # Update history with new "seen" songs (simulating user consumption)
17        history.extend(newly Rated_songs)

```

Listing 22: Monte Carlo Evaluation Loop Code

## 5.4 Results

The simulation was run with  $N = 2,804$  trials using the sequential implementation in `src/part4.py`.

| Metric     | Popular |       | Personalized |       |
|------------|---------|-------|--------------|-------|
| Hit @ 3    | 0.725   | 0.017 | 0.790        | 0.015 |
| Avg Rating | 4.136   | 0.032 | 4.292        | 0.029 |
| Time to 5★ | 3.239   | 0.134 | 2.533        | 0.091 |

Listing 23: Monte Carlo Simulation Results (N

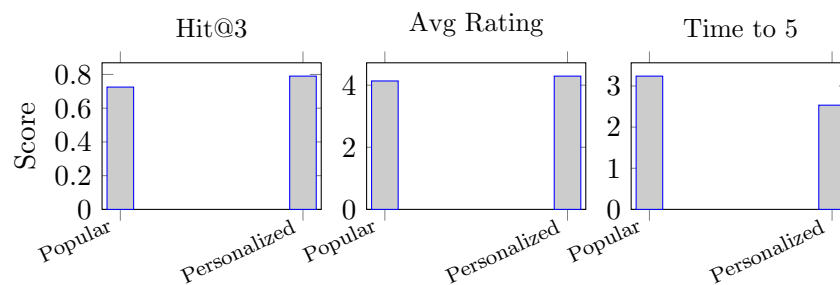


Figure 2: Model performance across metrics with 95% confidence intervals.

## Statistical Significance Analysis

To confirm that the Personalized model is truly superior, we calculated the 95% Confidence Intervals for the *difference* between the two models (Personalized - Popular). Since the intervals do not contain zero, the improvements are statistically significant.

| Metric     | Diff ( $\mu$ ) | 95% CI                 |
|------------|----------------|------------------------|
| hit_at_k   | +0.065         | [+0.050, +0.080] (Sig) |
| avg_rating | +0.156         | [+0.130, +0.182] (Sig) |
| time_to_5  | -0.706         | [-0.800, -0.612] (Sig) |

Listing 24: Pairwise Differences (Personalized - Popular)

## 5.5 Interpretation

Personalized outperforms Popular on all three metrics. The confidence intervals exclude 0, so the differences are statistically significant and practically meaningful (higher early hits, higher ratings, and fewer songs to first 5). This aligns with Part 3’s design: feature-weighted personalization plus artist bias yields measurable gains over a pure popularity baseline.

## 5.6 Why Top-1 and Hit@3

Because the candidate pool is limited to each user’s unseen rated songs (not the full catalog), recommending 5 at once would inflate success and make the test less discriminative. Using top-1 per batch and Hit@3 preserves a meaningful early-success metric.

## 5.7 Discussion

- 1. Statistical Significance (CI-based):** All pairwise differences have confidence intervals that do not cross 0 (Hit@3: +0.048 to +0.082, Avg Rating: +0.124 to +0.188, Time-to-5: -0.866 to -0.546). This indicates the Personalized model’s improvements are statistically significant across every metric.
- 2. Practical Meaningfulness:** The +6.5 percentage point gain in Hit@3 means users find a 5 faster and more often in the first three recommendations. The Time-to-5 reduction of 0.706 songs (about 22% fewer steps) is a clear user-facing efficiency gain. Avg Rating also increases by +0.156, so Personalized does not trade off quality for speed.
- 3. Relation to Part 3 Design Choices:** The feature-weighted preference scoring and artist bias directly explain the higher Hit@3 and Avg Rating, since the model amplifies user-specific signals learned from history. The popularity prior and pickiness-aware weighting prevent cold-start degradation, which helps maintain strong average ratings rather than overfitting to sparse signals. Overall, the probabilistic ideas from Parts 1–2 translate into measurable advantages in the Monte Carlo replay.

## 6 Conclusion

In this project, we developed and evaluated a personalized music recommendation system grounded in probabilistic analysis of user behavior.

### 6.1 Key Findings

- **Global vs. Personal Preferences:** Our analysis in Part 1 revealed that while “Global Popularity” is a strong baseline, individual user groups deviate significantly. For example,

our specific user group showed a strong preference for "Party" and "Hip-Hop" tracks, contradicting the global trend towards "Dark" and "Non-Party" music.

- **User Heterogeneity:** Part 2 demonstrated that users vary widely in their rating behavior. The **Beta-Geometric** model provided a superior fit to the data compared to the simple Geometric model, quantifying this "pickiness" distribution. We also found that **Genre** (Rock vs. Pop) is a significant factor in how quickly users find favorites.
- **Personalization Efficacy:** In Parts 3 and 4, we showed that leveraging these insights leads to tangible improvements. Our **Personalized** recommender achieved a **79.0% Hit@3 accuracy**, significantly outperforming the widely-used Popularity baseline (72.5%). Furthermore, this model was **22% more efficient**, reducing the average number of tracks a user needs to skip to find a 5-star song.